

УО «Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ПОИТ

Отчет по лабораторной работе №5

по предмету

Базы данных

Вариант 1

Выполнил:

Бражалович А.И.

Проверил:

Фадеева Е.Е.

Группа 351004

Минск 2025

Предметная область: Фирма по обслуживанию и ремонту компьютеров.

1. Определите, какие каскадные операции необходимы в этой базе данных. Настройте соответствующие каскадные операции.

- 1.1. ON DELETE CASCADE (Удаление родителя влечет удаление потомка)

Родительская таблица	Дочерняя таблица (связь М:М)	Причина
repair_order	repair_order_service	Если заказ удален, детали по услугам для этого заказа (строки) должны быть удалены.
repair_order	repair_order_part	Если заказ удален, детали по запчастям для этого заказа должны быть удалены.

- 1.2. ON DELETE RESTRICT (Запретить удаление родителя)

Родительская таблица	Дочерняя таблица (связь)	Причина
client (Клиент)	device (Устройство)	Нельзя удалить клиента, если у него есть привязанные устройства. Сначала нужно удалить устройства.
employee (Сотрудник)	repair_order (Заказ)	Нельзя удалить сотрудника, если он является менеджером или инженером в активных заказах.
service (Услуга)	repair_order_service	Нельзя удалить услугу из прайс-листа, если она используется в каком-либо заказе.

Родительская таблица	Дочерняя таблица (связь)	Причина
part (Запчасть)	repair_order_part	Нельзя удалить запчасть со склада, если она была использована в каком-либо заказе.
position_lookup (Должность)	employee	Нельзя удалить должность, пока на ней числятся сотрудники.
device_type_lookup (Тип устройства)	device	Нельзя удалить тип, если к нему привязаны устройства.

1.3. ON UPDATE CASCADE для BCEN внешних ключей.

- Создайте список представлений, которые нужно добавить в базу данных, а также подписок задач, выполняемых каждым представлением. Создайте соответствующие представления.

Название представления (View)	Основная задача	Задачи, выполняемые представлением
vw_active_orders_summary	Оперативная сводка по текущим ремонтам.	Предоставляет единую таблицу для быстрого обзора всех заказов в статусе «Принят» или «В работе», включая ФИО клиента, модель устройства, назначенного инженера и дату поступления.
vw_full_invoice_details	Детализация счета для	Объединяет данные по использованным

Название представления (View)	Основная задача	Задачи, выполняемые представлением
	выставления инвойсов.	запчастям (repair_order_part) и оказанным услугам (repair_order_service) в одном отчете для каждого заказа. Включает тип позиции, наименование, количество и фактическую цену.
vw_engineer_performance	Анализ эффективности и работы сотрудников.	Рассчитывает ключевые метрики для каждого инженера: общее количество завершенных заказов и среднее количество дней, затраченных на ремонт.
vw_inventory_status_and_value	Состояние складских запасов с аналитикой цен.	Показывает текущий остаток каждой запчасти, ее розничную и закупочную цену, а также вычисляет текущую общую стоимость запасов на складе (в розничных и закупочных ценах).

2.1. vw_active_orders_summary

```

CREATE VIEW vw_active_orders_summary AS
SELECT
    ro.order_id AS 'Order ID',

```

```

ro.status AS 'Статус',
ro.order_date AS 'Дата приема',
ro.planned_completion_date AS 'Планируемая дата завершения',
c.name AS 'Клиент',
c.phone AS 'Телефон клиента',
dtl.type_name AS 'Тип устройства',
d.model AS 'Модель устройства',
d.serial_number AS 'Серийный номер',
mgr.full_name AS 'Менеджер (Принял)',
eng.full_name AS 'Инженер (Назначен)'
FROM
    repair_order ro
JOIN
    device d ON ro.device_id = d.device_id
JOIN
    client c ON d.client_id = c.client_id
JOIN
    device_type_lookup dtl ON d.device_type_id = dtl.type_id
JOIN
    employee mgr ON ro.employee_id = mgr.employee_id --
Менеджер
LEFT JOIN
    employee eng ON ro.engineer_id = eng.employee_id -- Инженер
(может быть NULL)
WHERE
    ro.status IN ('Принят', 'В работе');

```

2.2. vw_full_invoice_details

```

CREATE VIEW vw_full_invoice_details AS
SELECT
    ros.repair_order_id AS 'Order ID',
    'Услуга' AS 'Тип позиции',
    s.name AS 'Наименование',
    ros.quantity AS 'Количество',
    ros.actual_service_price AS 'Цена за ед.',
    (ros.quantity * ros.actual_service_price) AS 'Сумма'
FROM
    repair_order_service ros
JOIN
    service s ON ros.service_id = s.service_id

```

UNION ALL

SELECT

```
    rop.repair_order_id AS 'Order ID',  
    'Запчасть' AS 'Тип позиции',  
    p.name AS 'Наименование',  
    rop.quantity AS 'Количество',  
    rop.actual_selling_price AS 'Цена за ед.',  
    (rop.quantity * rop.actual_selling_price) AS 'Сумма'  
FROM  
    repair_order_part rop  
JOIN  
    part p ON rop.part_id = p.part_id;
```

2.3. vw_engineer_performance

CREATE VIEW vw_engineer_performance AS

SELECT

```
    e.full_name AS 'Инженер',  
    COUNT(ro.order_id) AS 'Завершено заказов',  
    AVG(DATEDIFF(ro.actual_completion_date, ro.order_date)) AS  
    'Среднее время ремонта (дни)'
```

FROM

employee e

JOIN

repair_order ro ON e.employee_id = ro.engineer_id

WHERE

ro.status = 'Выдан' -- Только завершённые заказы

GROUP BY

e.employee_id, e.full_name

ORDER BY

'Завершено заказов' DESC, 'Среднее время ремонта (дни)' ASC;

2.4. vw_inventory_status_and_value

CREATE VIEW vw_inventory_status_and_value AS

SELECT

p.article AS 'Артикул',
p.name AS 'Наименование',
p.stock_quantity AS 'Остаток на складе',
p.purchase_price AS 'Закупочная цена',
p.selling_price AS 'Розничная цена',
(p.stock_quantity * p.purchase_price) AS 'Общая закупочная стоимость',
(p.stock_quantity * p.selling_price) AS 'Общая розничная стоимость'

FROM

part p

ORDER BY

p.stock_quantity ASC;

3. Создайте список проверок для добавления в базу данных вместе с подписком задач, выполняемых каждой проверкой. Создайте соответствующие проверки.

Название проверки (Constraint)	Таблица	Поле	Задачи, выполняе мые проверкой
chk_price_positive	service, part	standard_price, selling_price	Гарантируе т, что цены и стоимости не могут быть отрицатель ными, предотвращ ая логические ошибки в расчетах.

Название проверки (Constraint)	Таблица	Поле	Задачи, выполняе мые проверкой
chk_actual_price_po sitive	repair_order_s ervice, repair_order_p art	actual_service_p rice, actual_selling_p rice	Гарантируе т, что фактически е цены, использова нные в заказе, всегда положитель ны.
chk_dates_consistenc y	repair_order	order_date, actual_completi on_date	Обеспечива ет, что дата фактическо го завершения заказа не может быть раньше даты его приема в ремонт.
chk_quantity_used	repair_order_s ervice, repair_order_p art	quantity	Гарантируе т, что количество услуг или запчастей, использова нных в заказе, всегда больше нуля.
chk_engineer_for_in _progress	repair_order	engineer_id	Бизнес- правило: Заказ, который

Название проверки (Constraint)	Таблица	Поле	Задачи, выполняе мые проверкой
			находится в статусе «В работе» (В работе), должен иметь назначенного инженера (engineer_id IS NOT NULL).

3.1. Таблица **service**

ALTER TABLE service

ADD CONSTRAINT chk_service_price_positive CHECK (standard_price > 0);

3.2. Таблица **part**

ALTER TABLE part

ADD CONSTRAINT chk_part_selling_price_positive CHECK (selling_price > 0),

ADD CONSTRAINT chk_part_purchase_price_positive CHECK (purchase_price > 0);

3.3. Таблица **repair_order_service**

ALTER TABLE repair_order_service

ADD CONSTRAINT chk_os_actual_price_positive CHECK (actual_service_price > 0),

ADD CONSTRAINT chk_os_quantity_used CHECK (quantity > 0);

3.4. Таблица **repair_order_part**

ALTER TABLE repair_order_part

```
ADD CONSTRAINT chk_op_actual_price_positive CHECK  
(actual_selling_price > 0),
```

```
ADD CONSTRAINT chk_op_quantity_used CHECK (quantity > 0);
```

3.5. Таблица **repair_order**

```
ALTER TABLE repair_order
```

```
ADD CONSTRAINT chk_dates_consistency CHECK (actual_completion_date  
IS NULL OR actual_completion_date >= order_date);
```

```
ALTER TABLE repair_order
```

```
ADD CONSTRAINT chk_engineer_for_in_progress CHECK (  
    status != 'В работе' OR engineer_id IS NOT NULL  
);
```

4. Создайте список триггеров, которые нужно добавить в базу данных, а также подписок задач, выполняемых каждым триггером. Создайте соответствующие триггеры.

4.1. Триггер для автоматического заполнения цены услуги

```
CREATE TRIGGER trg_ros_set_default_price  
BEFORE INSERT ON repair_order_service  
FOR EACH ROW  
BEGIN  
    -- Если фактическая цена не задана (NULL или 0), берем  
    стандартную цену из справочника service  
    IF NEW.actual_service_price IS NULL OR  
    NEW.actual_service_price = 0 THEN  
        SELECT standard_price INTO NEW.actual_service_price  
        FROM service  
        WHERE service_id = NEW.service_id;  
    END IF;  
END
```

4.2. Триггер для автоматического заполнения цены запчасти

```
CREATE TRIGGER trg_rop_set_default_price  
BEFORE INSERT ON repair_order_part  
FOR EACH ROW  
BEGIN
```

```

-- Если фактическая цена не задана (NULL или 0), берем
розничную цену из справочника part
IF NEW.actual_selling_price IS NULL OR
NEW.actual_selling_price = 0 THEN
    SELECT selling_price INTO NEW.actual_selling_price
    FROM part
    WHERE part_id = NEW.part_id;
END IF;
END

```

4.3. Списание со склада при добавлении запчастей в заказ

```

CREATE TRIGGER trg_rop_stock_decrease_insert
AFTER INSERT ON repair_order_part
FOR EACH ROW
BEGIN
    UPDATE part
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE part_id = NEW.part_id;
END

```

4.4. Корректировка склада при изменении количества в заказе

```

CREATE TRIGGER trg_rop_stock_update
AFTER UPDATE ON repair_order_part
FOR EACH ROW
BEGIN
    -- Если количество ИЗМЕНИЛОСЬ
    IF OLD.quantity <> NEW.quantity THEN
        UPDATE part
        SET stock_quantity = stock_quantity + (OLD.quantity -
NEW.quantity)
        WHERE part_id = NEW.part_id;
    END IF;
END

```

4.5. Возврат на склад при удалении запчасти из заказа

```

CREATE TRIGGER trg_rop_stock_increase_delete
AFTER DELETE ON repair_order_part
FOR EACH ROW
BEGIN

```

```

UPDATE part
SET stock_quantity = stock_quantity + OLD.quantity
WHERE part_id = OLD.part_id;
END

```

5. Создайте список хранимых подпрограмм, которые будут добавлены в базу данных, а также подписок задач, выполняемых каждой подпрограммой. Создайте соответствующие подпрограммы.

Название подпрограммы	Тип	Основная задача	Задачи, выполняемые подпрограммой
sp_create_new_order	PROCEDURE	Полный цикл приема нового заказа	1. Проверяет существование клиента. 2. Создает новую запись в таблице device. 3. Создает новую запись в таблице repair_order со статусом 'Принят'. 4. Возвращает ID нового заказа.
sp_update_order_status	PROCEDURE	Контролируемое обновление статуса заказа	1. Обновляет поле status в repair_order. 2. Бизнес-правило: Если статус меняется на 'Выдан', автоматически устанавливает actual_completion_date = CURDATE(). 3. Бизнес-правило: При переводе в 'В работе', проверяет наличие

Название подпрограммы	Тип	Основная задача	Задачи, выполняемые подпрограммой
			назначенного инженера (дублирует CHECK-constraint).
sp_replenish_part_stock	PROCEDURE	Ручное пополнение запасов на складе	1. Проверяет, что part_id существует. 2. Увеличивает stock_quantity на указанное количество. 3. Можно использовать для регистрации закупок.
sf_get_monthly_revenue	FUNCTION	Финансовая аналитика: месячная выручка	Рассчитывает общую выручку (сумму total_amount) по всем заказам, которые были выданы (status = 'Выдан') за указанный месяц и год.

5.1. sp_create_new_order

```

CREATE PROCEDURE sp_create_new_order(
    IN p_client_id INT,
    IN p_device_type_id SMALLINT,
    IN p_manufacturer VARCHAR(100),
    IN p_model VARCHAR(100),
    IN p_serial_number VARCHAR(100),
    IN p_problem_desc TEXT,
    IN p_manager_id INT,

```

```

        IN p_planned_date DATE,
        OUT p_order_id INT
    )
BEGIN
    DECLARE v_device_id INT;

    -- Проверка существования клиента и менеджера
    IF NOT EXISTS (SELECT 1 FROM client WHERE client_id =
p_client_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Ошибка: Клиент не найден.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM employee WHERE
employee_id = p_manager_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Ошибка: Менеджер не найден.';
    END IF;

    START TRANSACTION;

    -- 1. Добавление устройства
    INSERT INTO device (client_id, device_type_id, manufacturer,
model, serial_number, receipt_date, description_of_problem)
    VALUES (p_client_id, p_device_type_id, p_manufacturer,
p_model, p_serial_number, CURDATE(), p_problem_desc);

    SET v_device_id = LAST_INSERT_ID();

    -- 2. Добавление заказа
    INSERT INTO repair_order (device_id, employee_id, order_date,
planned_completion_date, status)
    VALUES (v_device_id, p_manager_id, CURDATE(),
p_planned_date, 'Принят');

    SET p_order_id = LAST_INSERT_ID();

    COMMIT;
END

```

5.2. **sp_update_order_status**

```

CREATE PROCEDURE sp_update_order_status(

```

```

        IN p_order_id INT,
        IN p_new_status ENUM('Принят', 'В работе', 'Готов', 'Выдан',
'Отменен'),
        IN p_engineer_id INT
    )
BEGIN
    -- Проверка на "В работе" без инженера
    IF p_new_status = 'В работе' AND p_engineer_id IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Ошибка: Для статуса "В работе" необходимо назначить
инженера.';
    END IF;

    -- Обновление заказа
    UPDATE repair_order
    SET
        status = p_new_status,
        engineer_id =
            CASE
                WHEN p_new_status = 'В работе' AND p_engineer_id IS
NOT NULL THEN p_engineer_id -- Назначаем инженера, если
статус "В работе"
                ELSE engineer_id
            END,
        actual_completion_date =
            CASE
                WHEN p_new_status = 'Выдан' THEN CURDATE() --
Автоматически проставляем дату выдачи
                ELSE actual_completion_date
            END
    WHERE order_id = p_order_id;
END

```

5.3. sp_replenish_part_stock

```

CREATE PROCEDURE sp_replenish_part_stock(
    IN p_part_id INT,
    IN p_quantity INT
)
BEGIN
    IF p_quantity <= 0 THEN

```

```

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
        'Ошибка: Количество для пополнения должно быть
        положительным.';
    END IF;

    -- Обновление количества
    UPDATE part
    SET stock_quantity = stock_quantity + p_quantity
    WHERE part_id = p_part_id;

    -- Проверка, был ли обновлен какой-либо ряд (т.е. существует
    ли запчасть)
    IF ROW_COUNT() = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
        'Ошибка: Запчасть с указанным ID не найдена.';
    END IF;
END

```

5.4. **sf_get_monthly_revenue**

```

CREATE FUNCTION sf_get_monthly_revenue(
    p_year INT,
    p_month INT
)
RETURNS DECIMAL(15, 2)
READS SQL DATA
BEGIN
    DECLARE v_revenue DECIMAL(15, 2);

    SELECT COALESCE(SUM(total_amount), 0.00) INTO v_revenue
    FROM repair_order
    WHERE status = 'Выдан'
        AND YEAR(actual_completion_date) = p_year
        AND MONTH(actual_completion_date) = p_month;

    RETURN v_revenue;
END

```